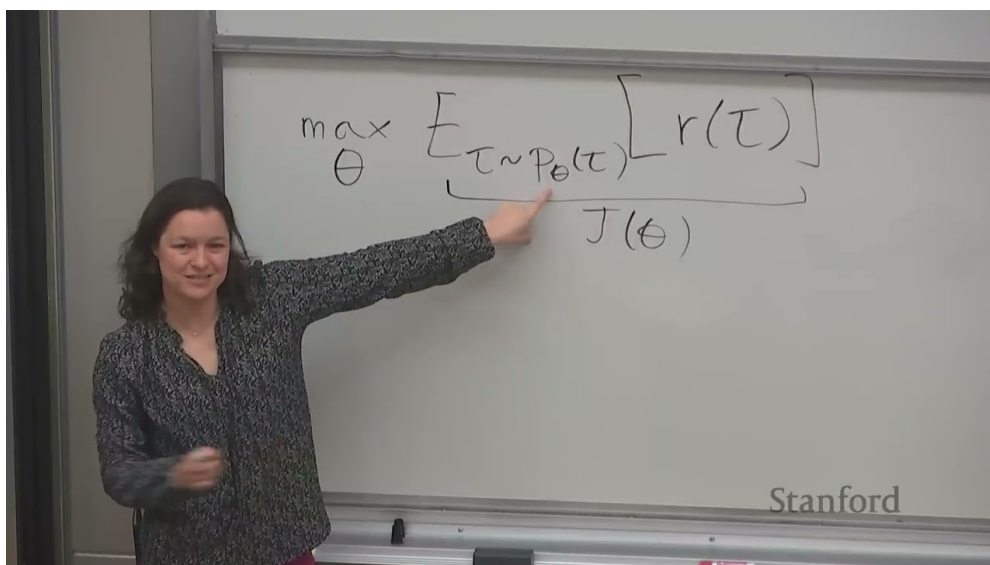


课程笔记

CS224R Lecture 3: 策略梯度

基于 Stanford CS224R 2025 公开课程整理

2026 年 4 月 4 日



视频作者/频道: Stanford Online

发布日期: 2025-04

视频时长: 约 80 分钟

讲者: Chelsea Finn

课程主页: cs224r.stanford.edu

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 从模仿学习到强化学习	3
1.1 问题空间与能力跃迁	3
1.2 强化学习的优化目标	3
1.3 Online RL 的基本流程	3
1.4 本章小结	3
2 策略梯度的推导	3
2.1 目标函数与轨迹因式分解	3
2.2 Log-Gradient Trick	4
2.3 Monte Carlo 近似: REINFORCE	4
2.4 本章小结	4
3 直觉理解与改进	5
3.1 按奖励加权的模仿学习	5
3.2 Reward-to-Go 与因果性	5
3.3 Baseline 与 Advantage	5
3.4 代理目标与自动微分	6
3.5 本章小结	6
4 方差控制与样本效率提升	6
4.1 GAE 与 temporal smoothing	6
4.2 Entropy 正则与 Trust Region	6
4.3 本章小结	7
5 系统约束与稳定性保证	7
5.1 资源与算力限界	7
5.2 制度与公平	7
5.3 本章小结	8
6 策略梯度的实现与调试	8
6.1 伪代码与最佳实践	8
6.2 调试提醒	8
6.3 本章小结	8
7 策略梯度与 Actor-Critic 结合	9
7.1 Actor-Critic 架构导入	9
7.2 Actor-Critic vs 原始策略梯度	9
7.3 本章小结	9
8 Off-Policy 策略梯度	9
8.1 On-Policy 的局限	9
8.2 Importance Sampling	10
8.3 Replay Buffer 与 Truncated Importance Sampling	10

8.4	方差控制提醒	10
8.5	本章小结	10
9	评估指标与 Benchmark	10
9.1	可靠性与可解释性指标	10
9.2	Benchmark Suites	11
9.3	本章小结	11
10	案例研究: CartPole 与 PPO	11
10.1	环境与任务介绍	11
10.2	训练流程与日志	11
10.3	复盘与调优	12
10.4	本章小结	12
11	扩展话题: 多任务与分布漂移	12
11.1	多任务 policy gradient	12
11.2	分布漂移与 domain adaptation	12
11.3	本章小结	13
12	合规、审计与工程落地	13
12.1	Logging 与可追溯性	13
12.2	工程交付 checklist	13
12.3	本章小结	13
13	总结与延伸	13
13.1	总结表	13
13.2	延伸阅读	14
13.3	本章小结	14

1 从模仿学习到强化学习

1.1 问题空间与能力跃迁

上一讲介绍了模仿学习：给定专家演示，通过监督学习来学习策略。但模仿学习的两个核心瓶颈依然存在：无法超越演示者、缺乏自我强化的练习机制。强化学习引入 reward 的概念，把行为放入长期决策的背景中，因此可以通过大量体验逐步提升。

从 Offline 到 Online 的差异

- **Offline**: 只能重用历史数据，适合无法自由交互的场景（模拟器/医疗）。
- **Online**: 能够不断采集新数据，允许策略在实际环境中探索。

策略梯度属于 online RL，它把行为策略直接当作优化变量。

1.2 强化学习的优化目标

强化学习中的终极目标是最大化 trajectory reward: $J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[\sum_t r_t]$ 。我们不再尝试拟合专家动作，而是在 θ 空间上做梯度下降（或上升），使得策略更倾向于高 reward 的轨迹。

为什么不直接做监督学习？

- 行为数据来自不同策略的分布，不能假设 IID；
- reward 提供了从头探索的反馈，能发现超越专家的策略；
- RL 中的 π_{θ} 决定数据采集，策略和数据是联合变化的。

1.3 Online RL 的基本流程

1. 初始化 π_{θ} （随机、模仿、启发式）。
2. 运行 π_{θ} 采样多个 trajectory（collect data）。
3. 估算策略梯度并更新参数 θ 。
4. 用新策略继续采样，形成闭环。

1.4 本章小结

本章从问题形式和能力跃迁角度强调：策略梯度与模仿学习的关系在于采样分布的更新、奖励目标的显式建模，以及在更广泛策略空间内寻找高 reward 解的能力。

2 策略梯度的推导

2.1 目标函数与轨迹因式分解

策略梯度最大化的目标函数是 trajectory reward 的期望：

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, a_t) \right],$$

其中 $p_\theta(\tau) = p(s_1) \prod_t \pi_\theta(a_t | s_t) p(s_{t+1} | s_t, a_t)$ 。我们把策略参数 θ 视为唯一的优化对象，其余项仅决定 sampling distribution。

轨迹因式分解的意义

- 动力学部分 $p(s_{t+1} | s_t, a_t)$ 与 θ 无关，因此不影响梯度；
- 策略 π_θ 决定了如何采样， $J(\theta)$ 也是对采样策略的期望；
- reward 在整个轨迹上累积，形成 a global signal，需用 differential form 拆开。

2.2 Log-Gradient Trick

利用恒等式 $\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$ ，可以写出：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} [\nabla_\theta \log p_\theta(\tau) \cdot r(\tau)].$$

展开 $\log p_\theta(\tau)$ 得： $\log p(s_1) + \sum_t [\log \pi_\theta(a_t | s_t) + \log p(s_{t+1} | s_t, a_t)]$ ，其中只有 $\log \pi_\theta$ 依赖参数，从而得到：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\left(\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \right) \cdot r(\tau) \right].$$

无需知道环境模型

环境动力学 $p(s_{t+1} | s_t, a_t)$ 被 log-gradient trick 消掉了，因此我们可以在 model-free 环境中直接采样轨迹并估计目标。

2.3 Monte Carlo 近似：REINFORCE

在实际中，用 Monte Carlo estimate 替代期望：

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_t \nabla_\theta \log \pi_\theta(a_{i,t} | s_{i,t}) \right) \cdot r(\tau_i).$$

该公式即 REINFORCE 的梯度估计器。算法流程是：(1) 采样 N 条 trajectory；(2) 计算每条轨迹的 reward；(3) 用上述估计更新参数。

符号	含义	备注
$\sum_t r(s_t, a_t)$	轨迹总 reward	轨迹级 supervision
$\nabla_\theta \log \pi_\theta$	策略对数似然梯度	可自动微分
N	采样条数	影响 variance、compute

表 1: REINFORCE 估计器的关键量

2.4 本章小结

策略梯度的核心是把 trajectory reward 期望写成 log-likelihood $\times$ reward 的形式，从而在不建模动力学的前提下直接估计梯度。REINFORCE 给出了最原始的 Monte Carlo 实现，但 variance 较大，后续章节讨论改进手段。

3 直觉理解与改进

3.1 按奖励加权的模仿学习

策略梯度实际上是**按轨迹奖励加权的模仿学习**：增强高 reward 的动作，抑制低 reward 的动作。相比纯监督，这种写法允许我们通过 stochastic policy 做探索。

试错学习的本质

- 策略生成采样，reward 决定 gradient 的方向；
- 更灵活地处理稀疏 reward，因为采样中包含了整个轨迹；
- 优化目标是期望 reward，而不是单步准确率。

3.2 Reward-to-Go 与因果性

我们希望某个 time step 的梯度只受该 step 及之后 reward 的影响。用 reward-to-go 可以实现因果性：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right).$$

该表达式在每个动作处只观测未来 reward，所以更符合 causal effect。

术语	含义	作用
Trajectory reward	整个轨迹累加 reward	全局 supervision
Reward-to-go	当前时刻到终点的 reward	保持因果性
Baseline	常数或估值函数	减少 variance

表 2: 策略梯度估计中常用的 reward 表达

3.3 Baseline 与 Advantage

对奖励尺度敏感

如果所有奖励都是正的，梯度会增加所有动作的似然——即使有些动作只是“没那么差”。这会显著增加梯度方差。

通过从 reward-to-go 中减去 baseline $b(s_t)$ ，我们不改变期望但可以降低方差：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (G_t - b(s_t)) \right],$$

其中 G_t 是 reward-to-go。若 $b(s_t) = V^{\pi}(s_t)$ ，得到 advantage estimator $A^{\pi}(s_t, a_t)$ 。

Advantage 的直觉

Advantage 衡量某动作好于当前状态平均水平的程度。正 advantage 强化该动作，负 advantage 抑制。

3.4 代理目标与自动微分

将 reward-to-go 和 baseline 写入 surrogate objective:

$$\tilde{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \log \pi_{\theta}(a_{i,t} | s_{i,t}) \cdot \text{adv}_{i,t}.$$

该形式便于用自动微分计算梯度, 并且允许每条数据都同时参与多步优化, 而不必单独反向传播 $N \times T$ 次。

3.5 本章小结

本章从直觉角度解析策略梯度: 重点在于按奖励加权的模仿、因果性的 reward-to-go, 以及 baseline/advantage 的 variance 减少作用。代理目标函数使得梯度计算可以以标准方式在框架中实现。

4 方差控制与样本效率提升

4.1 GAE 与 temporal smoothing

Generalized Advantage Estimation (GAE) 在 reward-to-go 和 TD error 之间平滑切换:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l},$$

其中 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 。调节 λ 即可在 bias 和 variance 之间做 trade-off。

GAE 的调节逻辑

- $\lambda = 1$ 对应完全 Monte Carlo (高 variance / 低 bias);
- $\lambda = 0$ 对应 TD(0) (低 variance / 高 bias);
- 实际上取 $\lambda \in [0.9, 0.98]$ 经常能取得最优折衷。

4.2 Entropy 正则与 Trust Region

为了鼓励探索, 在 loss 上加入 entropy regularization:

$$\tilde{J}_{ent}(\theta) = \tilde{J}(\theta) + \beta \mathbb{E}[-\log \pi_{\theta}(a | s)].$$

Entropy term 防止策略过早确定而陷入 suboptimal mode。另一个常见做法是限制 KL divergence, 形成 trust region update (如 TRPO):

$$\max_{\theta} \mathbb{E}[\log \pi_{\theta}(a | s) \hat{A}] \quad \text{s.t.} \quad D_{KL}(\pi_{\theta_{old}} \| \pi_{\theta}) \leq \delta.$$

探索与稳定的平衡

- Entropy regularization 提供 soft constraint, 鼓励概率分布更平坦;
- KL constraint 通过 trust region 明确限制步长, 避免 policy collapse;
- 典型配对是 PPO (clipped objective) + entropy bonus。

4.3 本章小结

GAE、entropy boost 与 trust region 是在实际训练中提升稳定性和样本效率的常见手段，它们把 variance 控制在可管理范围并保持 sufficient exploration。

5 系统约束与稳定性保证

5.1 资源与算力限界

策略梯度训练常面对有限的算力与 budget：长 horizon 轨迹意味着巨大的 GPU 时间，而 entropy regularization 需要额外 compute。我们通常通过 accumulate gradients、梯度剪裁与 mixed precision 来平衡训练负载，并在 pipeline 中加入时序 dependency 检查防止 drift。

约束类型	具体表现	工程应对
延迟/预算	轨迹 horizon 长导致 compute burst	gradient accumulation + distributed rollout
稳定性	entropy collapse 造成 policy collapse	entropy bonus + KL guard
数据可用性	limited rollout data per day	replay buffer + prioritized sampling

表 3: 系统约束下策略梯度的资源格局

算力与 logging 的联动

- 每个 rollout 阶段都要写入 seed, config, entropy curve 以便复现；
- 长 horizon 轨迹在记录时需 chunking，避免 log 文件爆炸；
- Closed-loop pipeline 里把 gradient step count 与 runtime metric 绑定，便于 capacity planning。

5.2 制度与公平

除了硬件，模型部署还受到合规、公平与安全的制度约束。例如在 multi-domain rollout 中必须检测 policy 对不同群体的 reward 分布，如果差距过大需要 trigger guardrail。此处常用 fairness dashboard 记录 disparity, reject rate 与 manual override logs。

忽略制度要求的风险

如果只关注 reward 而忽略 fairness 指标，policy 可能在 release 后因差异化表现被 regulator 叫停，延缓整个产品进度。

制度化保障的做法

- 设立 ‘fairness gate’：对敏感 subgroup 的 reward/distribution 做提前评估；
- 对 ‘override event’ 进行 structured logging，关联 policy version 与 prompt template；
- 用 audit-ready benchmark（如 fairness-specific tasks）确保多维度对齐。

5.3 本章小结

系统约束既来自算力 budget, 也来自公平与 compliance 要求。通过细化 logging、entropy guard 以及 fairness gate, 可以把策略梯度训练拉回到可管理的工程轨道。

6 策略梯度的实现与调试

6.1 伪代码与最佳实践

下面是典型的 policy gradient 伪代码:

1. 启动环境, 初始化 θ , 准备 buffer。
2. 收集 N 条 trajectory, 记录 (s_t, a_t, r_t) 。
3. 计算 reward-to-go 与 baseline (可用 value network)。
4. 构造 surrogate loss, 执行一轮 gradient update。
5. 重复, 每 K 步 eval 一次策略。

组件	建议范围	说明
Batch size	2048–8192 steps	决定 variance 和 compute
Learning rate	1e-4 – 5e-4	小 learning rate 保证稳定
Entropy coeff	0.01–0.03	激励探索, 防止收敛太快
λ	0.9–0.98	GAE smoothing

表 4: 训练策略梯度时的关键超参数

6.2 调试提醒

忽略日志会错过差错

策略梯度容易发生梯度爆炸/消失、exploding reward scale、policy collapse。必须记录 reward curve、policy entropy、KL divergence, 否则难以定位问题。

调试建议

- 先从小环境 (CartPole) 验证数据 pipeline;
- 手动监控 advantage 分布, 确保其均值在零附近;
- 使用 gradient clipping 或 trust region 避免大步更新。

6.3 本章小结

实现层面需要把采样、估计、优化、日志串成闭环; 合理的超参、调试指标与记录机制是防止训练崩坏的保障。

7 策略梯度与 Actor-Critic 结合

7.1 Actor-Critic 架构导入

Actor-Critic 结构将策略 (actor) 与 value function (critic) 一起训练。Critic 提供 advantage 估计, actor 则根据 advantage 调整策略, 形成双塔结构:

1. Actor 产出 action distribution $\pi_{\theta}(a | s)$ 。
2. Critic 估计 $V_w(s)$, 供 advantage 计算 $\hat{A}_t = G_t - V_w(s_t)$ 。
3. Actor 更新使用 above advantage, Critic 用 TD loss 更新值网络。

Actor-Critic 的协同

Actor 根策略提供 high-level decision; Critic 判断 reward 估值, 两者共同缓解传统策略梯度的高 variance。

Critic 的 bootstrap update 用 $\delta = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$ 定义 TD error, 再最小化 δ^2 能让 critic 提供更准确的 baseline。

7.2 Actor-Critic vs 原始策略梯度

原始策略梯度仅使用 Monte Carlo reward, 因此 variance 大、数据利用差; Actor-Critic 则引入 value estimate 作为 baseline, 降低 variance 并实现 bootstrapping。

方法	估计类型	样本利用
REINFORCE	Fully Monte Carlo	每条轨迹使用一次
Actor-Critic	Bootstrapped value	同一数据可反复更新
PPO	Clipped surrogate + value head	利用多个 epoch

表 5: 策略梯度系列方法对照

在实际训练中, actor 的 gradient 表达式依旧是 $\mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a | s) \cdot \hat{A}]$, 但 advantage 由 critic 提供, 可用 GAE_n 形式提高稳定。Critic 更新时, 可通过多步 TD (如 TD(λ)) 融合 Monte Carlo 和 bootstrap 信号, 在双塔架构中形成协同。

7.3 本章小结

Actor-Critic 把策略梯度与 value function 结合, 降低了 variance 并提升了样本效率; PPO/TRPO 进一步在此基础上加入 trust region, 形成更稳定的 training loop。

8 Off-Policy 策略梯度

8.1 On-Policy 的局限

Vanilla policy gradient 是完全 on-policy 的: 每次梯度更新后, 之前的数据就不能再用了。这意味着每个梯度步都需要重新收集数据, 效率很低。

8.2 Importance Sampling

可以用 importance weights 来复用旧策略 π_θ 收集的数据来更新新策略 $\pi_{\theta'}$:

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_\theta(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - b \right)$$

Importance Sampling 不是万能的

当新旧策略差异很大时, importance weights 的方差会非常大, 使得估计不可靠。因此 importance sampling 只适用于策略非常接近的情况。

8.3 Replay Buffer 与 Truncated Importance Sampling

为了进一步提高数据利用率, 可以把过去的 trajectory 存入 replay buffer, 结合 truncated importance sampling (例如 PPO 的 clips / ratio clipping) 来约束 weight:

$$\hat{g} = \min \left(\frac{\pi_{\theta'}(a | s)}{\pi_\theta(a | s)}, 1 + \epsilon \right) \nabla_\theta \log \pi_\theta(a | s) \hat{A}.$$

Clip 机制限制了 ratio 的过度增长, 防止过大的 importance weight 导致梯度爆炸。

Clip ratio 的作用

- 限制 ratio 在 $[1 - \epsilon, 1 + \epsilon]$ 之间, 避免 over-optimistic update;
- 仍保留了 off-policy 的复用能力, 只要新旧策略保持相似。

8.4 方差控制提醒

除了 ratio clipping, 也可以 anneal ϵ 、增加 entropy bonus、加入 trust region gate。更复杂的做法是把每条 trajectory 关联的 importance weight 归一化, 使得多样本 update 时不会被某一条高 weight 数据主导。

8.5 本章小结

Vanilla policy gradient 是 on-policy 的, 数据利用率低。Importance sampling 可以实现一定程度的 off-policy 更新, 但受限于新旧策略的相似度。

9 评估指标与 Benchmark

9.1 可靠性与可解释性指标

策略梯度需要在多个维度上评估: 不仅仅是平均 reward, 还包括 policy entropy、KL divergence、advantage distribution 的稳定性。我们常用以下指标:

- **Reward Curve**: 单位时间或 epoch 的平均 reward;
- **Entropy**: 衡量 policy 的多样性, 避免 collapse;
- **KL Divergence**: 新旧策略之间的差异, drop-out 可用于 early stopping;

- **Advantage Signal**: monitor advantage 的均值与方差，确保 training signal 对称。

忽略多维指标的风险

只看 reward 曲线会错过 update 中的 mode collapse。必须在训练日志中同时记录 entropy + KL，从数据科学角度判断稳定性。

9.2 Benchmark Suites

CS224R 中常用的 benchmark 包括 CartPole、Pendulum 及 MuJoCo Control tasks。每个 benchmark 能力点不同：CartPole 强调 quick convergence，Pendulum 强调 continuous control，MuJoCo 强调 long-horizon stability。

Benchmark 的选择逻辑

- 用简单环境（CartPole）调参，确保 pipeline 无 bug；
- 过渡到更复杂（MuJoCo），检查 entropy 与 KL 变化；
- 统一记录 config & seed 以便 reproducibility。

9.3 本章小结

评估策略梯度不只看 reward，还要 monitor entropy + KL + advantage。Benchmark 环境提供了分级的测试集，可以在不同阶段收集调试 signal。

10 案例研究：CartPole 与 PPO

10.1 环境与任务介绍

CartPole 是经典的 balancing 问题：策略需要控制小车使得杆保持直立，reward 通常为 survival time。每条轨迹长度有限，适合 policy gradient 的在线收敛展示。

10.2 训练流程与日志

我们用 PPO 进行训练，采样 2048 步，A2C style 可能在本阶段失败。训练日志典型包含：total reward、policy entropy、clip fraction、value loss。

Metric	Interpretation
Clip fraction	探测梯度是否常 hit clip threshold
Entropy	policy 变化度
Value loss	Critic 是否收敛
Reward std	策略的鲁棒性

表 6: CartPole PPO 训练中常用指标

10.3 复盘与调优

如果 reward 停滞，可通过增加 entropy coefficient、减小 learning rate、增加 horizon 进行复盘。每次调优后记录 prompt & hyper parameters，并附上 ‘override note’ 以便 future engineer 复现。

复盘步骤

- 检查 reward curve 是否 Plateau;
- 观察 advantage 分布与 value loss;
- 调整 entropy / clip threshold, 记录 seed;
- 执行 soft reset (将 critic 权重设置为 previous checkpoint), 观察是否恢复。

10.4 本章小结

CartPole + PPO 提供了完整 pipeline: 从采样、估计、更新、到日志/复盘。此案例可作为大型 benchmark 的 micro 模型供新人快速掌握 policy gradient training。

11 扩展话题：多任务与分布漂移

11.1 多任务 policy gradient

在 multi-task RL 中，policy gradient 需要在不同任务间共享参数。最常见做法是使用 shared encoder + task-specific head, 这允许在不同 reward function 之间快速切换。训练时通常加入 task-id embedding 以区分 contexts。

多任务训练要点

- 同一 batch 中混合多个任务，若 reward scale 不同需标准化;
- 使用 task embedding 让 policy aware of active task;
- Critic 可以共享 backbone，但多头 value head 用于分别估计 advantage。

11.2 分布漂移与 domain adaptation

在实际部署时，训练数据与线上数据之间的差异会导致 policy drift。常见缓解方式包括：加入 domain randomization、使用 adversarial buffer 采样 hard cases、在 rollout 中自动扩充 replay buffer (cohort of policies)。我们需在 log 中明确记录 ‘data shift ratio’ 与 ‘reward shift ratio’。

忽略 domain shift 的后果

忽略分布漂移会让策略在小改动（如 sensor noise）下性能崩溃，纪要到 compliance review 可能会被判定为 “lacks robustness”。

11.3 本章小结

多任务训练与分布漂移是延展策略梯度到实际系统后的关键挑战，需在 policy architecture + logging pipeline 上做足手脚。

12 合规、审计与工程落地

12.1 Logging 与可追溯性

每个 policy update 应该伴随日志：seed、hyper parameters、KL divergence、critic loss。结论应该链接到 ‘audit note’ 以便 regulator 文件化 assessments。

Artifact	内容
Prompt registry	记录 prompt template + temperature
Override log	记录 clinician override + rationale
Evidence patch bank	存储 attention map 与 highlight frames
Release ticket	包含 rollout plan + rollback criteria

表 7: Policy gradient release 时的 audit artifacts

12.2 工程交付 checklist

每次 release 前必须 complete:

1. 冻结 prompt + hyper parameters;
2. 复核 KL divergence < threshold;
3. 归档 reward curve + entropy log;
4. 记录 fail-safe policy + rollback plan。

Release checklist 的价值

- 确保每次 update 都有可追溯的决策路径;
- 加速 regulator review 过程;
- 形成持续改进的 knowledge base。

12.3 本章小结

合规与 audit 是 policy gradient 落地的最后一公里：日志、artifact、checklist 构成了 regulator 可接受的 evidence trail。

13 总结与延伸

13.1 总结表

以下表格汇总本讲的主要模块、核心概念与实践重点，为复盘与 audit 提供直接的检索路径：

模块	核心概念	工程/实践要点
模仿 $\rightarrow$ 强化	Online RL + 采样闭环	策略参数直接控制数据分布
策略梯度推导	log-gradient trick	只优化 π_θ , 无需懂动力学
直觉与改进	reward-to-go + baseline	advantage 估计降低 variance
稳定性技术	GAE + Entropy + Trust Region	PPO/TRPO + entropy bonus
实现细节	超参 + 调试指标	monitoring reward curve 和 entropy
Off-policy	Importance weights	仅在策略近似时复用旧数据

表 8: 按模块划分的关键输出与风险控制

13.2 延伸阅读

- Williams, *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning* (REINFORCE 原文, 1992)。
- Schulman et al., *High-Dimensional Continuous Control Using Generalized Advantage Estimation*。
- Sergey Levine, *CS285 Lecture on Policy Gradients*: 另一套策略梯度推导与工程技巧。
- Schulman et al., *Proximal Policy Optimization Algorithms*: 理解 trust region/clipped objective 的落地方法。
- Sutton & Barto, *Reinforcement Learning: An Introduction* (第 13 章) 涵盖 policy gradient 与 actor-critic。

13.3 本章小结

本讲围绕策略梯度的形式推导、直觉解释与工程实践展开, 强调 variance control、entropy regularization 以及实现层面的 logging/checkpoint 策略。Off-policy 的讨论指出了该系列方法的数据瓶颈, 为下一步引入 actor-critic 与 value-based hybrid 亮出舞台。